RMC Engineering Competition

Programming Competition Package

Operation Vanguard

2026–2027 Academic Year · English Version

Design a distributed defence. Preserve the network. Prove your decisions.

Team: four engineering students

Development period: seven hours

Final assessment: 50% simulator performance · 30% engineering submission · 15% presentation · 5% resource management

Contents and competition at a glance

Section	Page
1. Introduction	3
2. Problem statement	3
3. Project specifications	4
3.5 Mission outcomes and simulator scoring	5
4. Deliverables	6
5. Assessment and official evaluation	7
5.4 Artificial intelligence and permitted resources	8
6.1 Engineering submission rubric — 30 points	9
6.2 Presentation and resource rubric — 20 points	10
6.3 Judges' score sheet — 100 points	11

Your task

Write one Lua agent that runs independently on every router in the supplied Operation Vanguard simulator. Contain a simulated botnet, restore affected hosts, and keep required workstation-to-server services working. The simulator, dashboard, Lua API, and handbook are supplied; you are not being asked to build a new simulator or graphical interface.

What you submit

- A runnable Lua entrypoint, all required helper modules, and a short execution README.
- A two-page engineering report and a concise, reproducible test-results file.
- Presentation slides for a maximum 10-minute presentation followed by 5 minutes of questions.
- A sources and AI-use declaration, including any organizer examples you adapted.

First 30 minutes

1. Open the supplied offline **VanMail.html** and read the briefing and simulator overview.
2. Extract the handbook's example ZIP so helper modules retain their paths.
3. Run Tutorial 1 with the supplied executable; inspect a task and its result.
4. Try the solo example, then Tutorial 2 and inter-router messaging.
5. Establish a baseline before changing your strategy. Ask **OCdt Syed or OCdt Gerami** for simulator clarification early.

Authority: this package governs competition assessment and submissions. The pinned simulator and published mission settings govern execution and numerical scores. VanMail documents the API. Organizers will distribute binding clarifications to every team; report any conflict before submission.

1. Introduction

1.1 About the competition

The RMC Programming Competition challenges a team of four engineering students to design, implement, test, and explain a working software solution using engineering design principles. Teams have **seven hours** of development time. The submission deadline and presentation schedule are announced at the opening briefing.

Operation Vanguard is a deterministic network-defence competition. Success requires balancing service continuity, infection containment, recovery, limited computation, and distributed knowledge. A simple, well-tested agent may outperform a more elaborate policy that cannot preserve service or explain its decisions.

Teams receive the simulator executable, the offline VanMail handbook, example agents and helper modules, tutorial/practice networks, and the official evaluation instructions. Organizers supply a common evaluation environment; competitors do not need to build the simulator or frontend during the competition.

2. Problem statement

2.1 Motivation and background

A network that has been disconnected is not necessarily a network that has been defended. During a botnet outbreak, users still require access to critical services. Blanket isolation may limit spread while denying the very services the defence exists to protect. Restoring a host without addressing continuing exposure may simply permit reinfection.

In this fictional evaluation, the command team is assessing Vanguard router modules for a distributed network. Each module has local authority and its own observations. Your team supplies the policy that tells those modules what to discover, which evidence to trust, when to contain a threat, and how to return hosts to useful service.

2.2 Objective

Develop a topology-independent Lua defence agent that:

- Discovers relevant hosts and uses task results to guide decisions.
- Preserves weighted required workstation-to-server service.
- Contains infection and removes established compromise.
- Returns recovered hosts online and safely releases unnecessary isolation.
- Uses bounded task slots, fuel, memory, and messaging effectively.
- Coordinates across routers when useful, without assuming shared Lua memory.

The engineering stakeholders are network users, critical-service owners, network operators, and the command team choosing a defence policy. Explain how your trade-offs serve those stakeholders. Performance evidence must come from actual simulator runs; planned behaviour and observed results are distinct.

3. Project specifications

3.1 Supplied interface and competitor responsibilities

Vanguard supplies Topology, Table, Globe, node inspection, Match score, Jobs, and telemetry. Use these tools to inspect decisions and gather evidence. Dashboard truth is available to the human operator; it is **not an input available to your Lua agent**. Discovered observations can be incomplete or stale. Globe locations are illustrative, not geographic network data.

You are assessed on your agent and engineering evidence, not the visual appeal of the supplied dashboard. Optional agent-authored debug drawings can explain decisions, but are not required and carry no standalone bonus.

3.2 Execution model and budgets

- The same submitted Lua sources execute in a private VM on every router. Globals, module caches, observations, memory, and fuel are private to each VM.
- Each router owns one LAN. Agents may manage their own hosts and router policies through the published API; coordination uses Vanguard messages or publish/subscribe.
- Each router has **five concurrent task slots**. Actions take fixed simulation-tick durations. Deploying a task is not the same as completing it; inspect status and handle failed results.
- Fuel meters Lua instructions and API/payload/task costs. Exhausting fuel can defer decisions to later ticks. Unused fuel does not shorten task durations or slow the botnet.
- World changes commit at deterministic barriers. Playback speed, pause, and browser rendering do not change the scoring rules. Restart reloads sources and creates a fresh run.

3.3 Network and threat model

Explicit TOML files define routers, LANs, workstations, servers, links, seeds, mission settings, and budgets. Routers are immune; workstations and servers can become compromised. Only workstations can start compromised. By default, infection takes **14 uninterrupted exposure ticks**, and an online compromised host can attack up to two targets.

Isolation or shutdown interrupts traffic but does not cure established infection. **RESTORE takes 20 ticks by default**, cures the host, and returns it online while retaining isolation and firewall rules. Same-LAN traffic bypasses router filters but still obeys endpoint filters. Inter-LAN traffic follows the simulator's fixed route and live policies; a blocked route does not automatically reroute.

3.4 Implementation requirements

Use the published **Lua API v1.3** and staged Lua modules inside the entrypoint's directory. No shell, filesystem inspection, external sockets, wall-clock access, or external decision service is available inside the sandbox. Derive topology and host IDs through permitted discovery/messages; do not submit fixture-specific infection lists or precomputed action schedules. Runtime use of discovered IDs is expected.

Official topology, mission rules, simulator code, and budgets must not be modified. Practice networks may be generated or edited for testing. Multi-router coordination is a design choice, not a mandatory complexity requirement.

3.5 Mission outcomes and simulator scoring

The simulator awards up to **1,000 points per scored run**. These become **50 points** in the competition result; the internal 600/250/150 weighting remains unchanged.

Simulator component	Maximum per run	What is measured	Competition equivalent
Service continuity	600	Weighted required connections working over the scoring horizon	30.0 / 100
Security	250	Fraction of non-router hosts free of established compromise over the horizon	12.5 / 100
Final operational recovery	150	Fraction of hosts clean, online, unisolated, and not blocked on any of their required service pairs at the finish	7.5 / 100
Total	1,000	Engine-reported terminal score	50.0 / 100

A required connection works only when both endpoints are clean and traffic is permitted, including endpoint power and relevant filters. Service weights come from the mission; when no pairs are supplied, the default is every workstation requiring every server. Routers are excluded from host fractions. A compromised host remains compromised when powered off. Exposure alone does not count as established compromise in the security fraction, but prevents verified victory.

Let **H** be the mission deadline, **s(t)** the working fraction of required service weight, **c(t)** the clean-host fraction, and **r(T)** the operational-host fraction at termination. Conceptually:

Run score = 600 × Σs(t)/H + 250 × Σc(t)/H + 150 × r(T).

The engine uses fixed-point accounting and truncation; its terminal component values are authoritative. Do not recalculate a different score from rounded dashboard percentages.

Terminal outcomes — default mission settings

- **VICTORY:** zero compromised hosts, no active exposure, every server serving at least one required client, and at least **90% of weighted required service** for **14 consecutive ticks**. Any failed condition resets the verification streak. Verified recovery stops the run immediately and freezes its state.
- **DEFEAT: 30 consecutive ticks with zero working required service**. Remaining horizon ticks earn no service/security credit, and final recovery earns zero.
- **INCOMPLETE:** the deadline is reached without verified recovery or outage defeat. The actual terminal operational fraction determines recovery points; incomplete does not automatically mean zero.

On early victory, unused horizon ticks receive the final service and clean-host fractions. Partial service stays partial credit. Dashboard recovery points are provisional until termination. Missions may explicitly publish different windows or thresholds; the official manifest records them.

Example: average service 80%, average clean hosts 90%, and final operational recovery 60% yield approximately **480 + 225 + 90 = 795 / 1,000**, or **39.75 / 50**. There is no separate reward for spare fuel, wall-clock speed, extra GUI features, or declaring success yourself.

4. Deliverables

Submit a single folder or ZIP through Google Classroom, with a repository link if requested by the organizers. Use your team name consistently. A link alone must identify an immutable commit or release that judges can access; the submitted archive is the frozen copy if both are provided.

4.1 Runnable agent and execution notes

- **agent.lua** as the entrypoint, plus every required Lua helper module with its relative path preserved. The same source set must run on every router and every official scenario without manual edits.
- **README.md** with team/member names, entrypoint, file layout, exact run commands, any permitted dependencies, and known limitations. The supplied executable is the runtime; no custom frontend build or simulator installation guide is required.
- **results.csv** or **results.md**, identifying each practice network/version, seed, budgets, tick limit, exact command, terminal tick/outcome, component scores, total score, and runtime errors. Identify baseline and changed agent revisions. Clearly label any truncated run as a smoke test, not a terminal result.
- **SOURCES.md** recording adapted organizer files, third-party sources, licences/attribution where applicable, and the AI-use declaration required in section 5.4.

Example execution from the submitted folder:

```
vanguard --script agent.lua --network network.toml
```

For evidence, use headless mode and `--replay results/run.json`; set `--ticks` to at least the scored mission's deadline. The CLI defaults to only 100 headless ticks, so omitting an adequate bound can produce an unfinished evaluation. A replay contains per-tick semantic hashes and a final snapshot, not a screen recording. Judges run the frozen submission independently.

4.2 Engineering report — maximum two pages

Use **Times New Roman, 11 pt, single-spaced, one-inch margins**. Cover stakeholder needs; measurable objectives; the chosen policy; sensing, task scheduling, containment/recovery, and coordination; alternatives and trade-offs; test evidence; limitations; and team design/development responsibilities. Include a compact state, flow, or sequence diagram when it clarifies the algorithm. UML is useful evidence, not a requirement to produce every diagram type.

Distinguish implemented behaviour from future work. Explain unsuccessful cases and one evidence-based improvement. Cite organizer examples and other reused material. Detailed test logs and a sources list may be separate files; do not use appendices to evade the two-page analysis limit.

4.3 Presentation

Submit Google Slides or a PowerPoint file named **[TeamName]-RMCEC-2026-Programming-Presentation.pptx** (or the corresponding named Google Slides link). Present for at most **10 minutes**, followed by **5 minutes of Q&A**. All team members must attend unless the director approves an exception. Explain the algorithm, justify key trade-offs, show representative evidence, and demonstrate the submitted agent. A short recorded run or replay evidence may support a live demonstration, but must be clearly identified.

5. Assessment and official evaluation

5.1 Overall result — 100 points

Component	Points	Evidence
Official simulator performance	50	Mean terminal score across the official run manifest
Engineering submission	30	Strategy, implementation, testing, and two-page report — section 6.1
Presentation	15	Technical explanation, evidence/Q&A, and delivery — section 6.2
Resource management and attribution	5	Reuse, citations, reproducibility handoff, and AI declaration — section 6.2
Total	100	Sum of the four components

For **N equally weighted official runs**, with engine terminal totals $S_1 \ldots S_n$:

Simulator points = $50 \times (S_1 + \ldots + S_n) / (1{,}000 \times N)$.

Example: scores 800, 900, and 700 average 800, contributing **40 / 50**. Add the rubric scores; do not rank by relative improvement over other teams. Use unrounded aggregate values for comparisons and display the final result to two decimal places. Outcomes are recorded separately; no additional victory bonus or outcome-based override is added to this formula.

5.2 Common evaluation conditions

At the opening briefing, organizers publish a run manifest specifying the pinned simulator version, official network/seed instances and mission deadlines, budgets, debug flag, run count, and equal weighting. Every team is assessed against that same manifest. If held-out instances are used, their count, family, and evaluation rules are announced before development; exact files may be withheld until assessment. Organizers freeze and record those instances before evaluating submissions.

Judges use an untouched copy of each official network and a fresh runtime for every run. Run to VICTORY, DEFEAT, or the published horizon; a human timeout is not a mission defeat. Capture the submitted-source identifier, manifest identity, command, component scores, outcome, terminal tick, and final hash. Machine or organizer faults are rerun under the same conditions; teams do not choose their best attempt. Repeated identical runs are verification, not extra scoring samples. Official debug settings are identical for all teams because instrumentation consumes fuel.

5.3 Consistent adjudication

Exact ties are broken by higher simulator points, then higher engineering-submission points, then higher presentation points; if still tied, share the placing. Judges must record evidence for manual marks. Do not deduct twice for the same weakness: a non-running agent earns no simulator points for an unexecutable official run, but assess any valid report/design evidence on its merits. Missing artifacts earn only the credit supported by submitted evidence. Presentation overtime is handled in the delivery criterion; stop at 10 minutes and begin Q&A.

The deadline freezes source and deliverables. Late changes require a director-approved, documented exception applied consistently; they are not silently substituted after judges see results. Verified plagiarism, prohibited AI-authored competition logic, or deliberate simulator/score tampering may result in disqualification after review and an opportunity for the team to explain. Automated AI-detection output alone is not proof.

5.4 Artificial intelligence and permitted resources

The competition assesses the team's own engineering decisions and agent logic. The following policy applies to the competition submission, including helper modules and generated fixes.

Use	Rule
Learn Lua or general programming concepts	Allowed: syntax explanations, conceptual research, and high-level algorithm discussion.
Understand public material	Allowed: explanations of VanMail, the published API, and supplied organizer examples.
Generate practice networks	Allowed: topology TOML/data using the public schema, samples, generator, or Network Agent.md. Never include competition agent code in the prompt.
Adapt organizer examples	Allowed: copy supplied examples and helper modules, cite them, understand them, and write your own changes.
Upload competition agent code	Prohibited: do not send your entrypoint, helpers, or excerpts to external LLM providers. This includes asking for debugging or review of your competition code.
Generate submitted agent logic	Prohibited: LLM-generated agent code, code completions, rewritten functions, generated fixes, or translated agent logic cannot be submitted.
Claim another source as your work	Prohibited: plagiarism or uncited copying. Citation does not make prohibited AI-generated competition code permissible.

External AI coding/completion features must not author the submitted agent. Research notes or explanations are not a substitute for understanding the algorithm. Network-generation permission applies to practice data, not modification of official evaluation inputs.

Source and AI-use declaration

In **SOURCES.md**, identify each organizer example or third-party code source you reused, the relevant files/functions, your modifications, and applicable attribution/licence obligations. State either "No AI tools used" or list the tools, purposes, and material supplied to them. For network generation, identify the generated network files. Do not fabricate citations or expose competition source in a prompt log.

Normal documentation, the organizer examples, and the supplied tooling are available to every team. Ask for clarification when uncertain about another external code source before using it; acknowledgment alone does not override a competition restriction. The simulator's existing dependencies are organizer-supplied infrastructure, not work your team must claim to have implemented.

Support and submission checks

OCdt Syed and OCdt Gerami provide OJE support for simulator behaviour, API interpretation, and setup questions. They may clarify the tools, not write or privately optimize a team's competition agent. Material rule clarifications are shared with all teams.

Before the deadline: run from a clean submitted folder; include every helper; verify a terminal practice result; check that the report is two pages; export the slides; record limitations; and complete the source/AI declaration. Keep an unchanged copy of the exact submitted revision for the presentation.

6. Appendix — analytic judging rubric

6.1 Engineering submission — 30 points

Use the submitted code, report, and reproducible evidence to select a score within each band. Record reasons for partial credit. Implementation quality is distinct from numerical performance; optional features earn no automatic bonus. Zero means missing or unrelated evidence.

Criterion / maximum	Strong	Competent	Developing	Limited / absent
Defence strategy and justification / 8	**7–8:** Coherent evidence-to-action policy; service, containment, recovery, and local/distributed knowledge trade-offs justified against alternatives and stakeholder needs.	**5–6:** Sound policy and main trade-offs explained; some alternatives, service consequences, or distributed assumptions lack justification.	**3–4:** Plausible policy, but largely descriptive; containment/release priorities or knowledge assumptions are weakly justified.	**0–2:** Little coherent strategy, major model misconceptions, or unsupported claims; no meaningful evidence at zero.
Implementation and robustness / 8	**7–8:** Inspectable state/task lifecycle; handles failures, discovery, ownership, slot limits and waits; avoids fixture-specific shortcuts; logic is understandable and consistent with the stated design.	**5–6:** Core lifecycle works; localized failure/stale-evidence or budget weaknesses remain; structure supports review.	**3–4:** Happy-path implementation exists but misses important task, recovery, knowledge, or failure cases; control flow is difficult to verify.	**0–2:** Fragmentary, largely unexplained, or seriously inconsistent implementation; no relevant submitted implementation at zero.
Testing and engineering iteration / 8	**7–8:** Reproducible baseline and changed-policy comparisons across multiple seeds/topologies; terminal components and errors recorded; adverse cases and limitations drive a documented change.	**5–6:** Reproducible baseline/improved runs and some variation; at least one limitation analysed; coverage or causal comparison is incomplete.	**3–4:** Some real runs, but narrow coverage, weak baseline, missing settings, or little analysis of why results changed.	**0–2:** Screenshots or anecdotal success only; truncated runs presented as conclusive; no verifiable tests at zero.
Two-page engineering report / 6	**6:** Clear stakeholder/problem definition, measurable objectives, accurate architecture/algorithm explanation, concise diagram where helpful, evidence-led conclusions, limitations and team responsibilities; required format respected.	**4–5:** Covers the problem, solution and evidence clearly; minor gaps in design explanation, limitations, responsibilities, or format.	**2–3:** Partial or poorly organized explanation; evidence and decisions weakly linked; substantial required content or format missing.	**0–1:** Minimal, misleading or unrelated report; absent report at zero.

Engineering subtotal: ____ / 30

6.2 Presentation and resource rubric

Presentation — 15 points

Criterion / maximum	Strong	Competent	Developing	Limited / absent
Technical explanation and design reasoning / 6	**6:** Accurate, accessible account of the submitted algorithm, state/task flow and distributed knowledge; explains alternatives and service/security trade-offs using relevant code or diagrams.	**4–5:** Correct explanation of major mechanisms and choices; a few details or trade-offs need prompting.	**2–3:** General overview with limited explanation of implementation or rationale; some model confusion.	**0–1:** Cannot explain core submitted logic, or no substantive technical explanation.
Demonstration, evidence and Q&A / 6	**6:** Demonstrates the frozen agent with traceable real results; distinguishes truth, observations, and claims; explains a failure/limitation and answers technical questions accurately.	**4–5:** Relevant authentic demonstration and mostly accurate answers; evidence traceability or failure analysis has minor gaps.	**2–3:** Limited demonstration or weak connection to the submission; answers reveal incomplete understanding of results or limitations.	**0–1:** No credible demonstration/evidence, materially misleading claims, or inability to address basic implementation questions.
Organization, timing and team contribution / 3	**3:** Logical, legible presentation within 10 minutes; all available members contribute meaningfully and explain their roles.	**2:** Clear overall delivery, but a minor timing, organization, legibility or contribution imbalance.	**1:** Major structure/time problems or unexplained participation imbalance; judge stops the presentation at 10 minutes.	**0:** No assessable presentation or contribution evidence. Approved attendance exceptions are accommodated without automatic loss.

Presentation subtotal: ____ / 15

Resource management and attribution — 5 points

Criterion / maximum	Strong	Competent	Developing	Limited / absent
Transparent reuse and reproducible handoff / 5	**5:** Complete source/AI declaration; reused files and modifications traceable; licences/credits addressed; README identifies the submitted revision, dependencies and exact execution procedure.	**3–4:** Honest and usable source/dependency records and run instructions, with minor omissions that do not obscure authorship or execution.	**1–2:** Sources or changes incompletely identified; AI declaration, dependencies or handoff instructions need substantial clarification.	**0:** No usable declaration or handoff evidence. Suspected misconduct is referred for review under section 5.3, not assumed from missing documentation alone.

Resource subtotal: ____ / 5

6.3 Judges' score sheet

Team number/name: _____ **Judge:** _____

Submitted revision/archive identifier: _____

Simulator version / official manifest: _____

A. Official performance record

For each manifest run, attach a row with: scenario and seed; deadline/budgets; terminal outcome and tick; service /600; security /250; recovery /150; total /1,000; runtime errors; final semantic hash; and any rerun reason. If more than one official run is used, keep every row; do not select a best run.

Official run count N: _____ **Sum of terminal run totals:** _____

Mean run score = sum / N: _____ / 1,000

Simulator contribution = mean × 0.05: _____ / 50

B. Analytic marks

Assessed criterion	Maximum	Awarded
Defence strategy and justification	8	
Implementation and robustness	8	
Testing and engineering iteration	8	
Two-page engineering report	6	
Engineering subtotal	30	
Technical explanation and design reasoning	6	
Demonstration, evidence and Q&A	6	
Organization, timing and team contribution	3	
Presentation subtotal	15	
Transparent reuse and reproducible handoff	5	
Resource subtotal	5	
Simulator contribution from A	50	
FINAL RESULT	100	

C. Evaluation checks and sign-off

- Same frozen source set, official inputs, simulator version, budgets and debug setting used for all applicable runs.
- Every scored run reached a mission terminal state; run failures, substitutions or organizer reruns documented.
- Component totals and conversion checked; manual marks have written evidence; no duplicate penalty applied.
- Submission/integrity questions referred to the director and any approved exception recorded.

Final result: _____ / 100 **Judge signature:** _____ **Date:** _____

Attach the per-criterion rationale, performance rows, and any exception/adjudication notes.